ELEC-H-310
Th06: Sequential logic circuits

Dragomir Milojevic
Université libre de Bruxelles

v3.1

Today

1. Race conditions & asynchronous circuits
2. Synchronous logic circuits
3. Flip-Flops & sequential circuits synthesis
4. Synchronous logic circuits synthesis
5. Example of sequential system synthesis

1. Race conditions & asynchronous circuits

Race conditions & asynchronous logic

- Problem of race conditions will appear when a state machine changes simultaneously more than two state variables at a time
  - ▹ Hamming distance between $y_{i,j}$ and $Y_{i,j}$ is > 1
- Race conditions are the consequence of physical implementation of a circuit: gates & wires introduce the delays due to real R, L, C values – parasitics; these delays in concurrent circuits will be different
- In sequential circuits with more than one variable (we do have one independent circuit per state variable), some circuits will be faster, the others will be slower
- Faster circuits may cause the system to end up in a wrong destination state, which will cause the system malfunction since for a given input sequence and departing state we end somewhere else
- Asynchronous sequential logic circuits solve race conditions by making sure that for every pair of source/destination states at a time we have only one state bit that changes at a time

Synthesis of asynchronous circuit implementation

- Final encoded state table can't contain critical race conditions
- To solve all races we use three different methods, applied in order to allow optimal system integration:
  - METHOD1 – State encoding → easiest & fastest
  - METHOD2 – Transitions modifications → introduces more constraints on the implementation, so could be less optimal than previous (e.g. don't cares used for transitions)
  - METHOD3 – Adding an extra state variable (to enable METHOD2) → worse in terms of logic ressources envolved
- Final step of the system synthesis concerns the output function – to avoid glitches we need to fix the values of the transitions so that only one change appears on the output
  - Exact moment when the output toggles is not important; what counts is the change of the output value

Asynchronous circuits – behind the scenes

- Every transition can be seen as one system cycle that takes a certain amount of time in circuits build using realistic components
- Stable state transitions (from one stable state to another) could have an arbitrary number of transitions, i.e. system cycles
- Abstracted circuits assume that the state transition is instantaneous, number of state transitions doesn't influence the transition time
- For an outside observer it is crucial that for every possible stable state transition we wait for necessary number of system cycles to:
  - Observe the output – if we do it earlier, we might read the old value and get the wrong output
  - We can not change the inputs during state transitions – we saw that this is mandatory to insure proper system operation (we don't want the system to end up somewhere else)
- Question is how to put the above in practice ...

Asynchronous circuits – the performance

- In any sequential machine the state transition will complete once we reach the destination stable state from the source stable state
- In an asynchronous system different stable state transitions could require different amount of system cycles (i.e. absolute time)
- System exhibits best possible performance since each stable state transition will take **exactly the amount of cycles (i.e. time) it needs to arrive to destination stable state**
- Consider state transition examples below:
  - Example 1: **00** $\rightarrow$ 01 $\rightarrow$ **01**
  - Example 2: **00** $\rightarrow$ 01 $\rightarrow$ 11 $\rightarrow$ **11**
- First stable state transition will be faster than the second because there is only one system cycle

From asynchronous to synchronous operation

- To make asynchronous logic operational, we have discretized the time using one system cycle for each state transition
- In the early years of digital electronics, practical systems have used the above approach, but it quickly became cumbersome for logic circuits that started to become more & more complex
- Luckily, in parallel, development of electronic components process technology enabled manufacturing of faster transistors, basic building blocks for logic gates implementation & quickly another idea emerged:

  Why not discretize the time for the complete duration of the stable state transition?

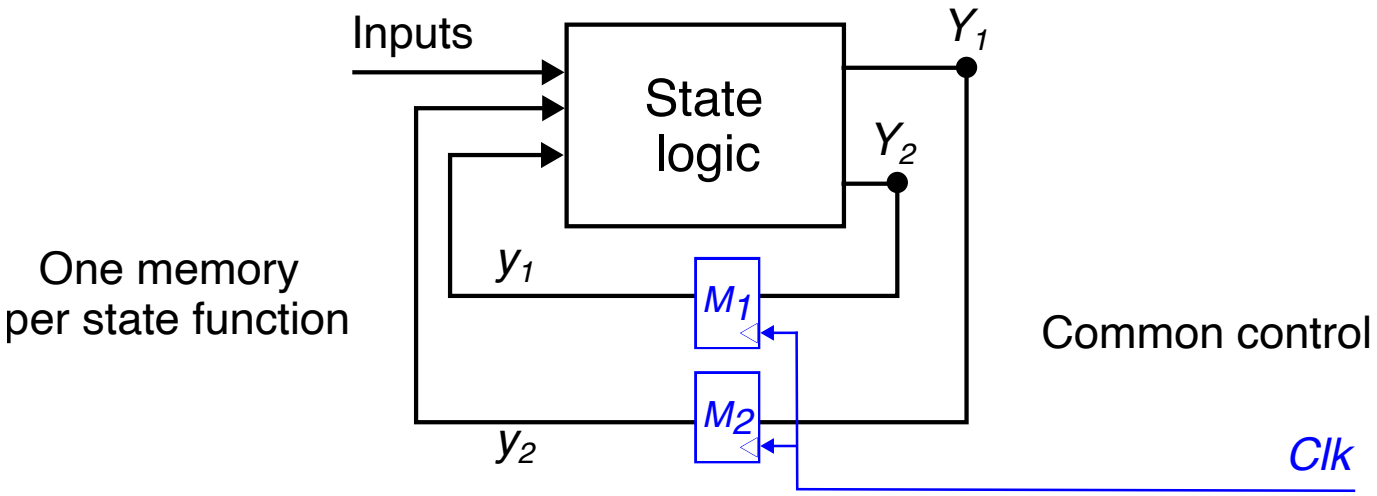
- If there are parts of the circuit that are faster, we could slow them down to match the speed of slower circuits
- And, if the time difference between the fastest and the slowest circuit is not that huge, the loss could be totally acceptable!

Welcome to synchronous logic circuits

2. Synchronous logic circuits

Controlled feedback delay → memory

- Problem of asynchronous circuits is in **different delays** of different state variable logic functions[1] – concurrent logic circuits
- To make all state variables delays **uniform**, we can use memories in the feedback circuitry, to store the state variables values
- State variable update will take place in regular (periodic) time intervals driven with an external control signal, common to all memories storing the state variables – clock *Clk*



[1] Delay will depend on logic and wiring delays

Synchronous circuits & their operation

- Because of the common *Clk* signal for all state memories we speak of synchronous operation & synchronous logic circuits
- Clock period will be sufficiently long to allow all system transients to happen at the input of the state memories
- Output of the memories will remain stable after it changes for the whole duration of the clock period
- State change occurs in two distinct steps
- Upon input change, first the future state is computed as function of inputs and present state:

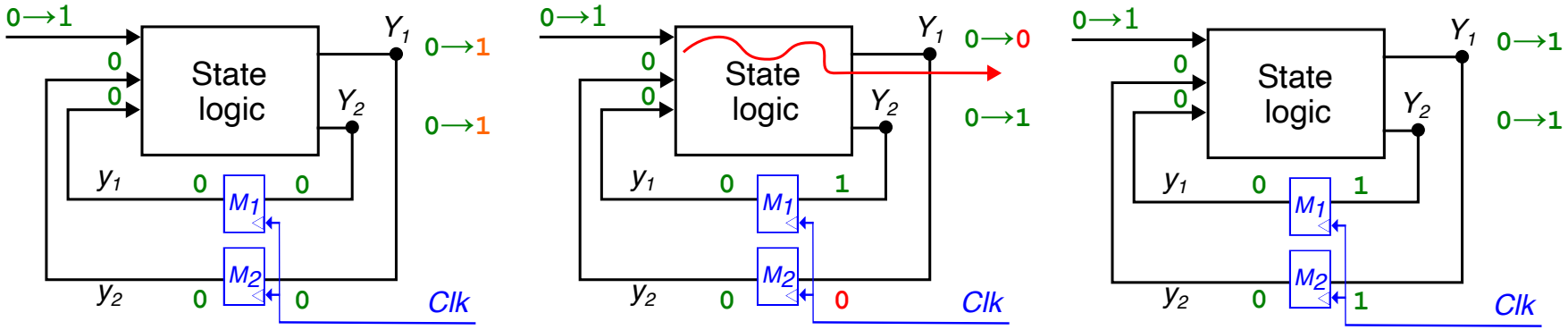
$$Y_i = f_i(I, y_i)$$

  and then, only after certain amount of time, future state is assigned to present state (memory update process):

$$y_i \leftarrow Y_i$$

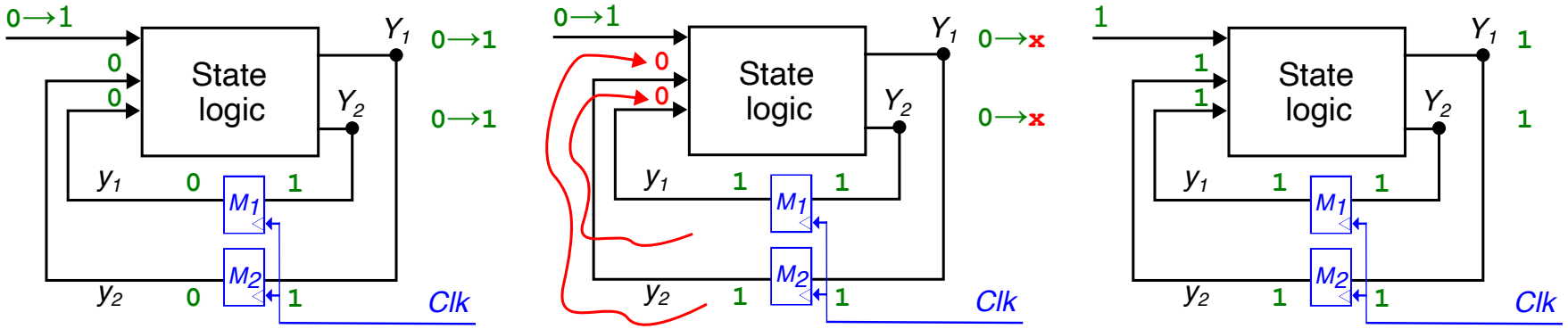
Synchronization: next state computation

- Example: we are in the stable state `00`; we remain there as long as input doesn't change; assume that the input changes from '0' to '1' and state logic computes `11` as next state (race condition)
- Due to logic and wire delays say Y_1 changes after Y_2; we will see transition `01` at the inputs of the memories, which is wrong
- But, if we wait for sufficient amount of time, the input to the memories will become stable `11`



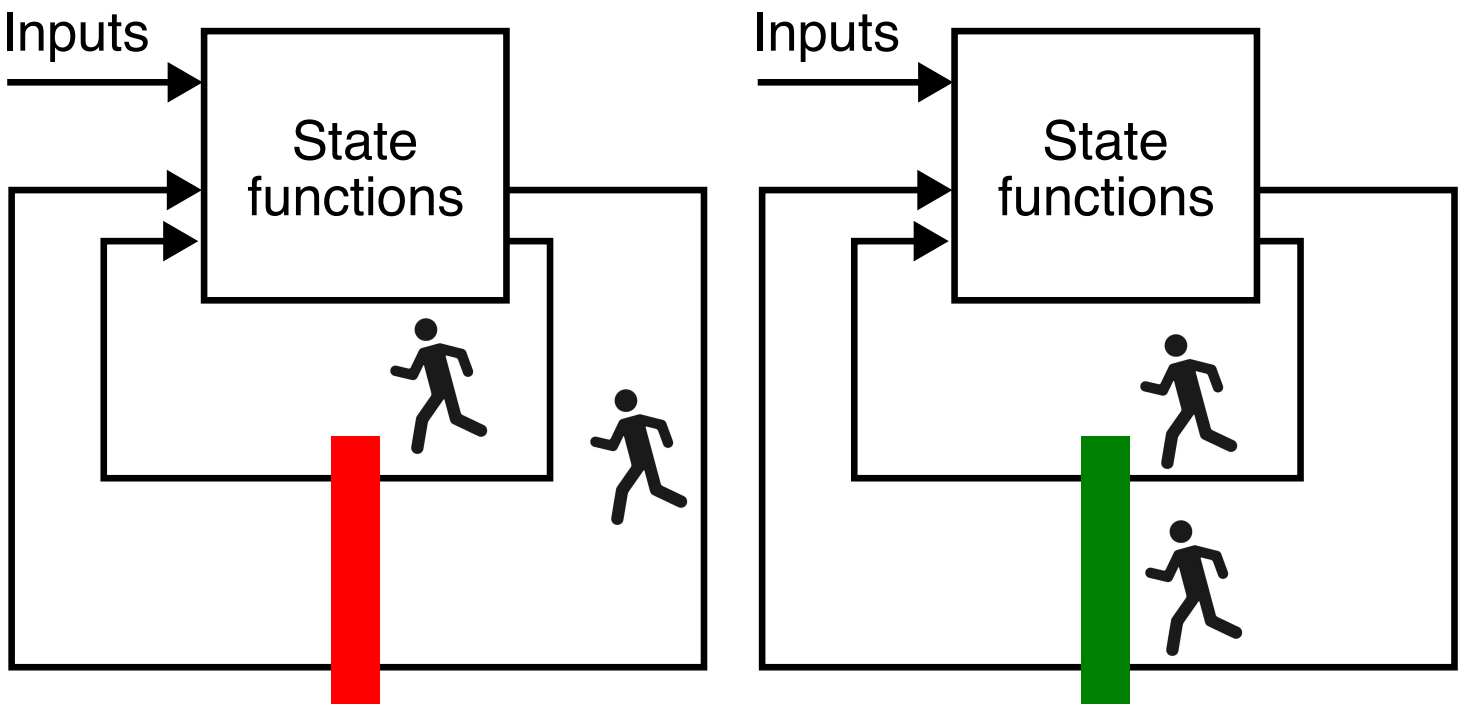
Synchronization: state update

- Note that during the input change, and the next state computation, memories kept the old (correct) state value of 00
- State value update, triggered by the rising edge of a *Clk* occurs only when we are sure that the input to memories is valid
- We will still have transients in the circuitry **after** the memories; but these will settle down if there is enough time & before the next rising edge of a *Clk*
  - ▹ Inputs must remain constant through the clock period (see later)



Synchronous circuits: race analogy

- Memories act as synchronization barrier that slow down the fastest signal, so that the slowest one can reach the memory
- In the below example racer in the inner loop arrives first, but will be halted at the level of the state memory
- Only once the second runner arrives, memories can be updated



Performance of sequential synchronous systems

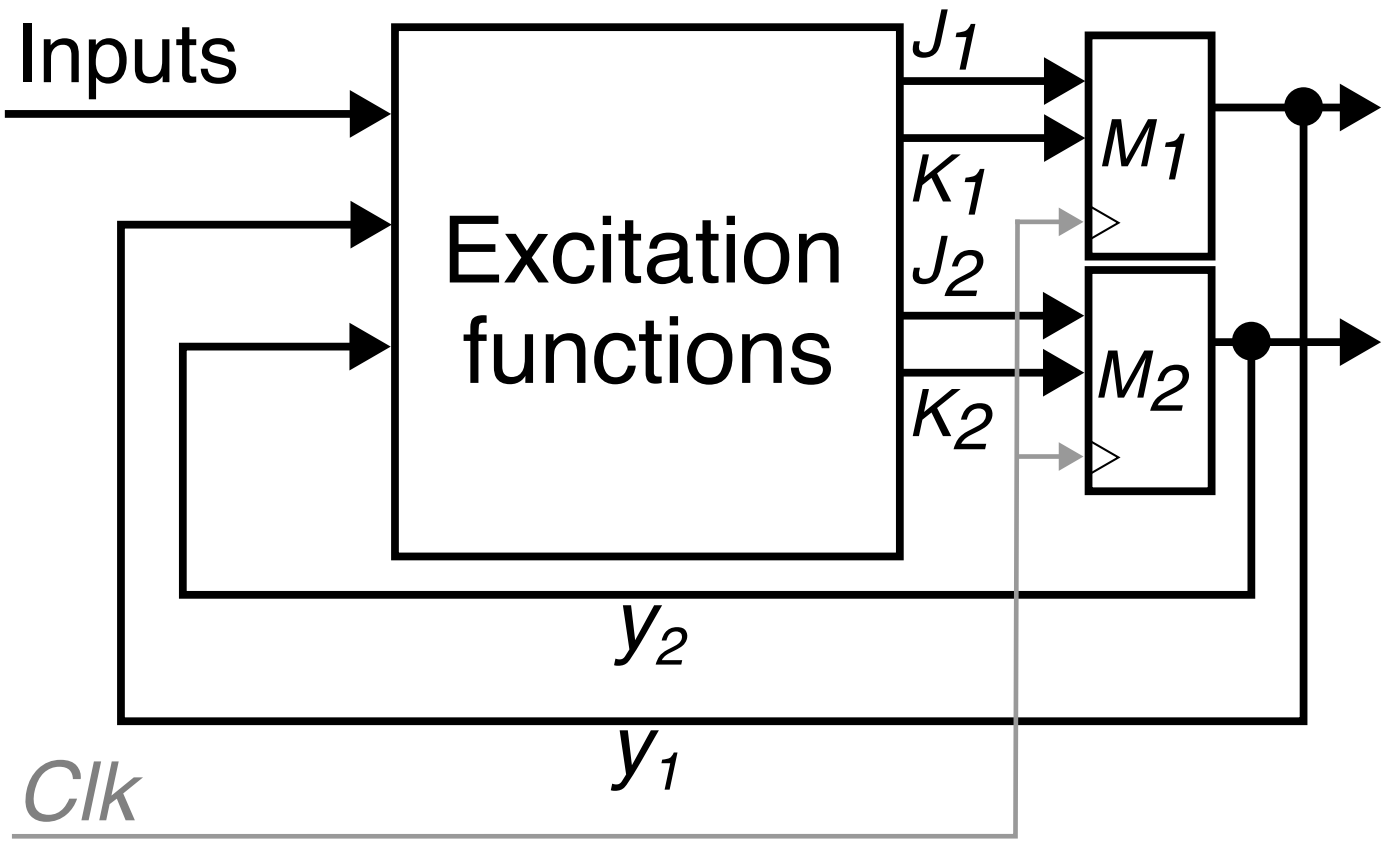
- The period of the control signal *Clk* is defined by the slowest signal path in the circuit, called critical path
- The system speed (we speak of a performance in case of circuits), i.e. how fast it can switch from one stable state to another, will be determined by this critical path
- Because of the synchronization, all other, even fast logic paths, will have to wait for the slowest one (necessary evil)
- This does not sound optimal! but could be accepted especially if the delay differences between different paths are not that big; in that case sacrifice we make is not that big
- Asynchronous systems will change state as quick as all transitions take place; absolute switching time from one state to another could be different for different source/destination states

Memory elements are Flip-Flop (FF)

- In previous slides we spoke about memory elements as a generic devices (we haven't said anything about how do they work); in real logic circuits, we will use Flip-Flops (FFs) as memory devices
  - ▹ One of the FFs: SR, JK, D or T
- These FFs will be driven by the control signals that should mimic the behavior defined by the state table
- FFs will store the actual state value, but at the input of FFs we will have to provide the appropriate control; these are called excitation functions
  - ▹ SR, JK will need two excitation functions and D, T only one
- The process of sequential synchronous circuit synthesis becomes the process of the excitation functions synthesis
- Because FFs are different, these functions will depend on the type of the FF used (FF type is the input of the problem)

Example for JK-FF

In the schematic below present can be observed (FF content); future is materialized through excitation functions $J_i K_i$

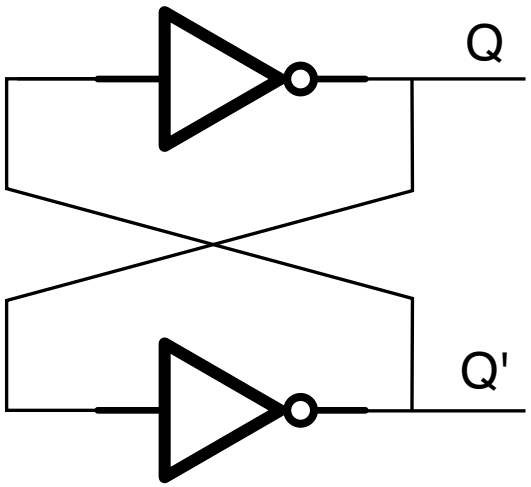


3. Flip-Flops & sequential circuits synthesis

Bistable circuits

- Bistable – simplest possible sequential circuit capable to memorize a single bit of information; different ways to build such circuit, figure below shows the implementation with a pair of inverters

- Output Q will take whatever the value will establish upon circuit power up
  - ▹ If '0' at the input of the top inverter $\rightarrow$ $Q = 1 \rightarrow Q' = 0$
  - ▹ If '1' at the input of the top inverter $\rightarrow$ $Q = 0 \rightarrow Q' = 1$
    - ◦ Similarly, assuming fixed '0' or '1' value at bottom inverter
  - ▹ Hence the name *bistable* – two possible states '0' or '1'



- Once established the above values will be maintained (saved), but we can't change the stored value (there are no inputs)

On latches & Flip-Flops (FFs)

- Depending on how the inputs will affect the output of the bistable, we differentiate two big classes of memory elements:
  - Latches – sensitive to the level of the input; whenever the input changes, the output will follow
  - Flip-Flops – the change of the output occurs only during the transition of the control signal from '0' to '1' – rising edge, or '1' to '0' – falling edge
- When we implemented our first sequential circuit we used edge triggered D-Flip-Flop for synthesis; edge being captured from the control signal C that if periodic becomes Clk
- Here we will discuss latches; behavior of the Flip-Flops is identical, if we do not show the control signal in the state tables; note that for actual synthesis of the synchronous systems we will be always using Flip-Flops even when Clk signal is not explicitly shown

Different memory element – state tables & graphs

SR

Q^+	00	01	11	10
0	**0**	**0**	–	1
1	**1**	0	–	**1**

Q

D

Q^+	0	1
0	**0**	1
1	0	**1**

Q

JK

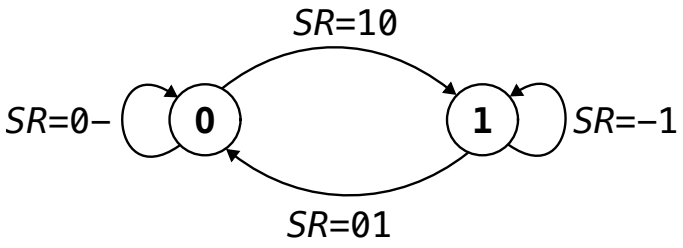
Q^+	00	01	11	10
0	**0**	**0**	1	1
1	**1**	0	0	**1**

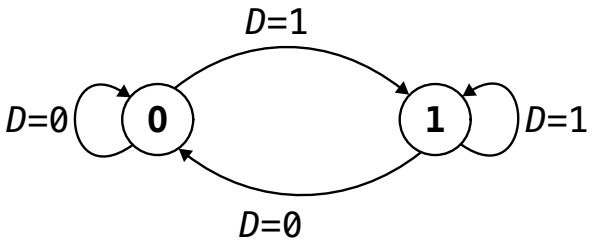
Q

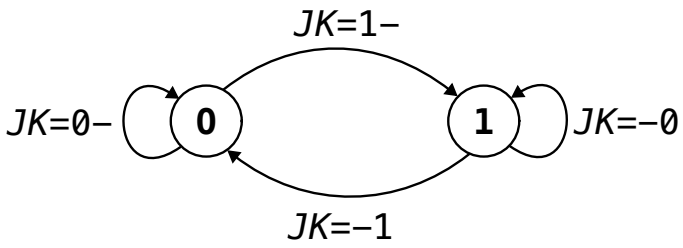
T

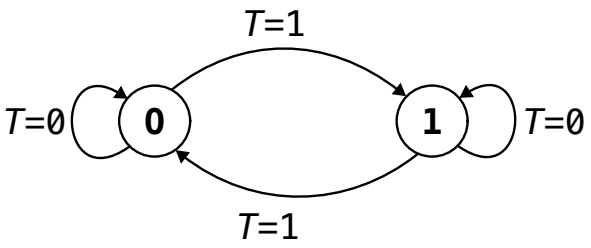
Q^+	0	1
0	**0**	1
1	**1**	0

Q









SR latch implementation with NAND

Q \ SR	00	01	11	10
0	**0**	**0**	–	1
1	**1**	0	–	**1**

q

$\Downarrow$

q \ SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

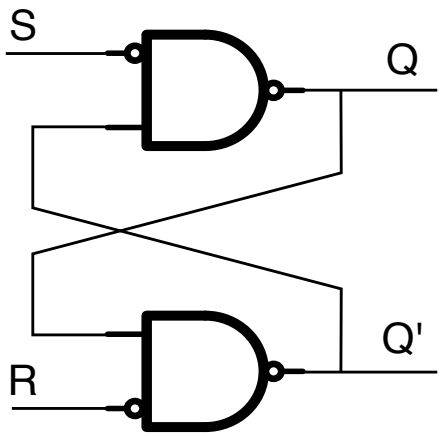
$$Q = S + qR' = \overline{\overline{S + qR'}}$$
$$= \overline{S' \cdot (\overline{q \cdot R'})}$$

$\Rightarrow$

q \ SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

$$Q' = R + q'S' = \overline{\overline{R + q'S'}}$$
$$= \overline{R' \cdot (\overline{q' \cdot S'})}$$

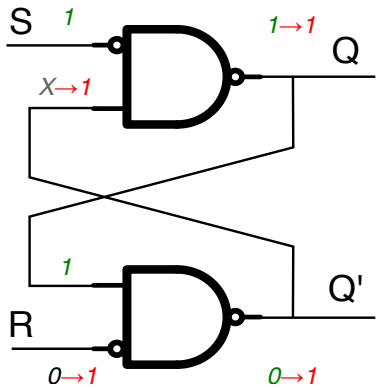
$\Downarrow$



On forbidden combination for SR

- If the inputs $SR = 11$, then both Q and Q' will be set to '1', this should not work one of them is wrong

$$
\begin{aligned}
Q &= S + qR' \\
Q' &= R + q'S'
\end{aligned}
$$



- In reality, the circuit will have delays, one feedback will arrive earlier; in the figure above we assume S has arrived first, so top gate switches and the Q is set to 1 (after certain amount of time Q' will set '1' and will be wrong); if you assume R to be the first you will have Q' set to '1' and Q wrong
- The problem is that we don't know in advance which one will switch first and thus we don't know the right value

Excitation tables for memory elements

- During synthesis of sequential circuits, instead of encoded state tables we will use the excitation tables
- They reflect the way how the excitation functions should change to allow given Boolean value to be stored in the memory element ('0' or '1')
- Four possibilities since we have present (Q) & future (Q^+) state and each each could have value of '0' or '1':

Q	Q^+	Operation	Code
0	0	Maintain 0	μ_0
0	1	Enable	ε
1	0	Disable	δ
1	1	Maintain 1	μ_1

State & excitation table for SR & JK

Q^+ \ SR	00	01	11	10
0	**0**	**0**	–	1
1	**1**	0	–	**1**

Q

Op.	Q	Q^+	S	R
μ_0	0	0	0	-
ε	0	1	1	0
δ	1	0	0	1
μ_1	1	1	-	0

Q^+ \ JK	00	01	11	10
0	**0**	**0**	1	1
1	**1**	0	0	**1**

Q

Op.	Q	Q^+	J	K
μ_0	0	0	0	-
ε	0	1	1	-
δ	1	0	-	1
μ_1	1	1	-	0

State & excitation table for D & T

D

Q^+	0	1
0	**0**	1
1	0	**1**

Q

Op.	Q	Q^+	D
μ_0	0	0	0
ε	0	1	1
δ	1	0	0
μ_1	1	1	1

T

Q^+	0	1
0	**0**	1
1	**1**	0

Q

Op.	Q	Q^+	T
μ_0	0	0	0
ε	0	1	1
δ	1	0	1
μ_1	1	1	0

4. Synchronous logic circuits synthesis

Algorithm 1/2

- Establishment of the primitive state table from verbal specs
  - Primitive state table has one stable state per line of the state table; we don't care about eventual sub-optimality at this step
  - Correct primitive state table is a must for a fully functional sequential logic circuit; any error at this step will result in a malfunctioning system; note that in this step we move from verbal ("loose") specification into a formal specification of a problem
  - All other synthesis steps are about model transformation that can be fully automated using computers
- Depending on the machine model adopted (Moore or Mealy) state table optimization produces optimized state table – minimal number of states to minimize the number of memory elements and excitation functions logic
- State encoding – because we don't care about race conditions, designer can chose any encoding, including the default one

Algorithm 2/2

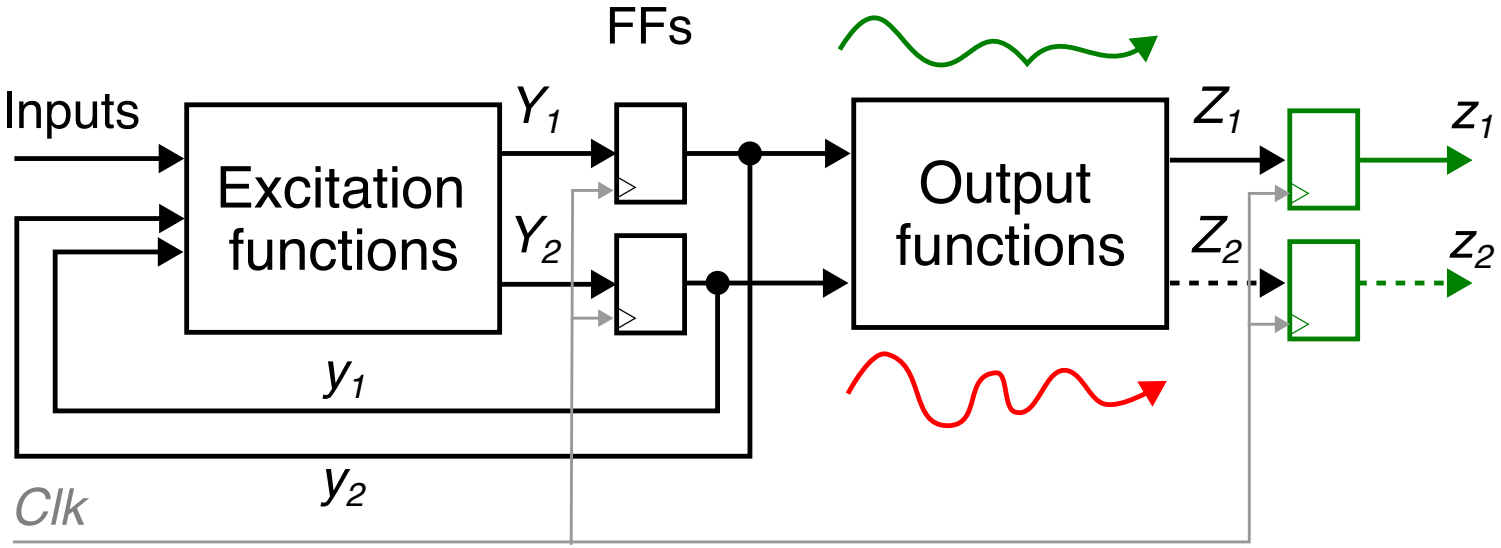
- From the encoded state table we derive excitation table of the logic system
  - Excitation table describes memory elements future (S^+), as function of the present (S) and input evolutions
  - Excitation table is independent from the FF choice
  - Since each memory element store a single bit (single feedback function) we have only 4 options: μ_0, μ_1, δ, ε
- Depending on the memory element choice (SR, JK, D or T) we derive truth table for each excitation function; K-Maps (or other methods) are used to derive optimal memory excitation functions
  - There will be two logic functions per state variable and for 2-inputs memory devices (SR, JK); and one logic function per state variable for single input memory devices (D, T)
- Synthesis of the system output – we will revisit this for the specific case of synchronous logic circuits

Sequential logic circuit synthesis

- Comparing asynchronous vs. synchronous synthesis algorithms:
  - Almost the same!
  - We only skip the step regarding race conditions, and we add synthesis of memory excitations functions depending on the choice of the memory element
  - State table can have race conditions
- Choice of a memory element could influence the complexity of the final circuit; we will not consider this level of optimization, memory element will be imposed
- We now have all elements required to do a complete system synthesis; let's do that on a concrete example (next section)
- But before the example let's re-consider the output synthesis in the case of synchronous operation

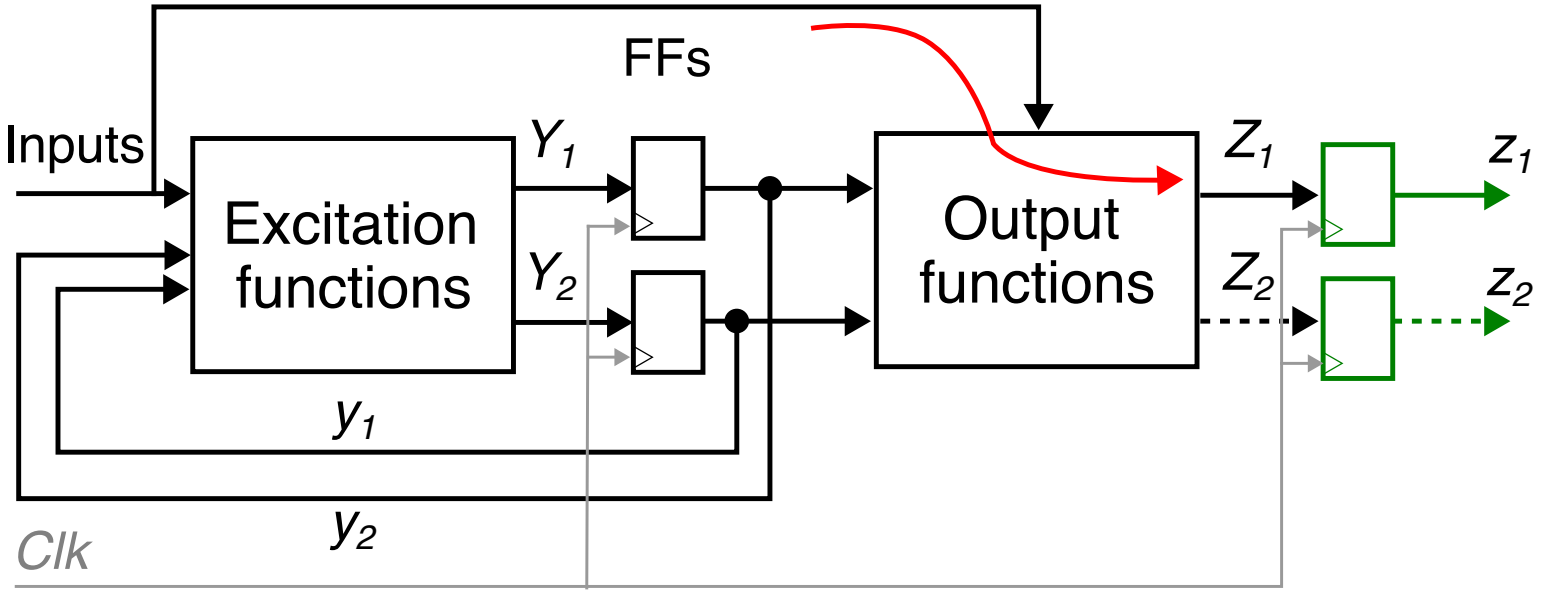
Output in synchronous circuits – Moore machine

- In Moore machine the output is function of state variables only
  - In asynchronous systems any transient in the state (feedback) circuitry will be visible on the output(s) – glitches; output transitions are fixed to a single change ('0' to '1' & '1' to '0')
- But now that we use memory elements to store the state, the output could be clean since it uses inputs from FFs; however in case of multiple outputs it is possible that one output switches earlier on
- To avoid output race, we can register each output, i.e. store it in a FF; we are now sure that we have synchronous & clean output
  - This comes at a cost → we need 2 clock cycles from input to output



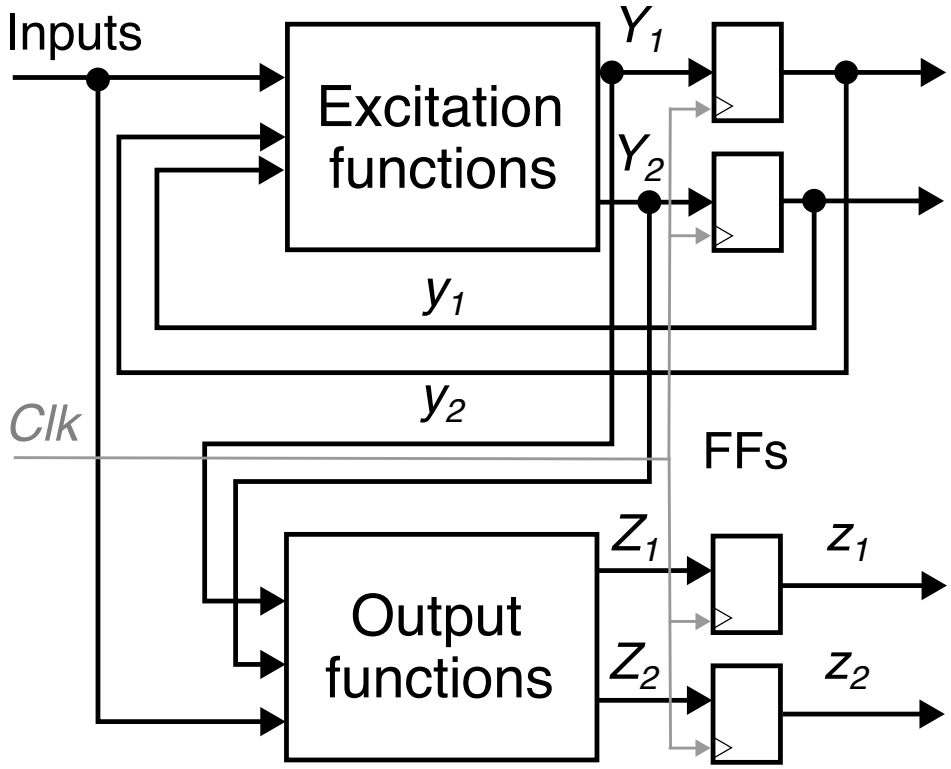
Output in synchronous circuits – Mealy machine

- In Mealy machine the output is function of & the input too
- Any glitch on the input will be propagated to output just after the output combinatorial logic (red curve)
- Once again registered output solves this at expense of two clock cycles latency; register output also allows Z_1 and Z_2 to remain valid for a full clock cycle



Reducing the output latency

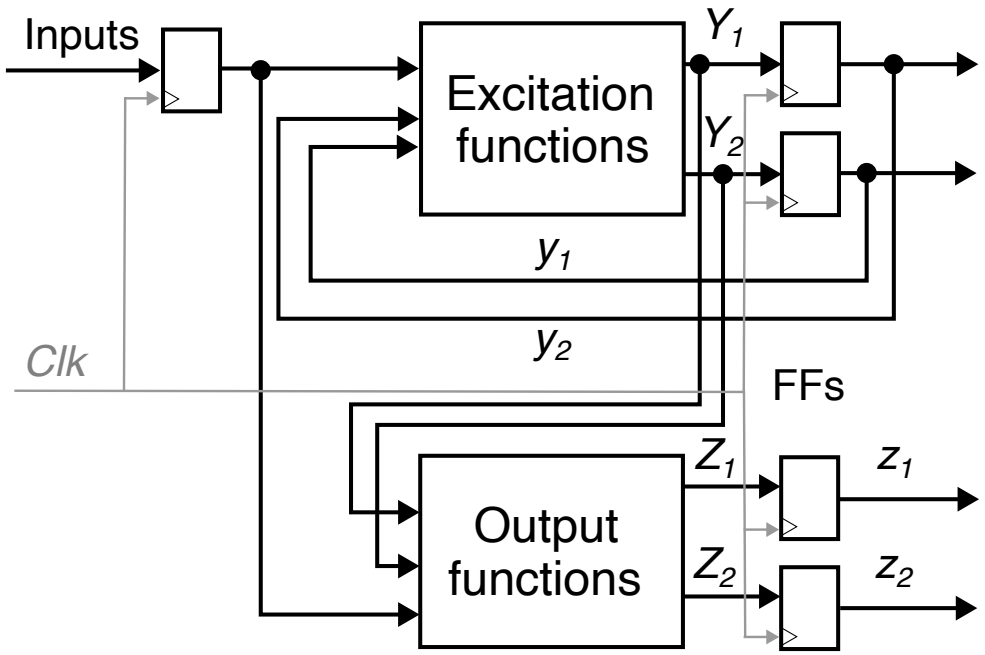
- Instead of state decoder it is be possible to synthesize the next state output as a function of future state & inputs
- Best of both worlds: clean output & 1 clock cycle latency



- But the problem could be that the critical path spans both excitation functions & output & affects period (to be checked)

Registering the inputs

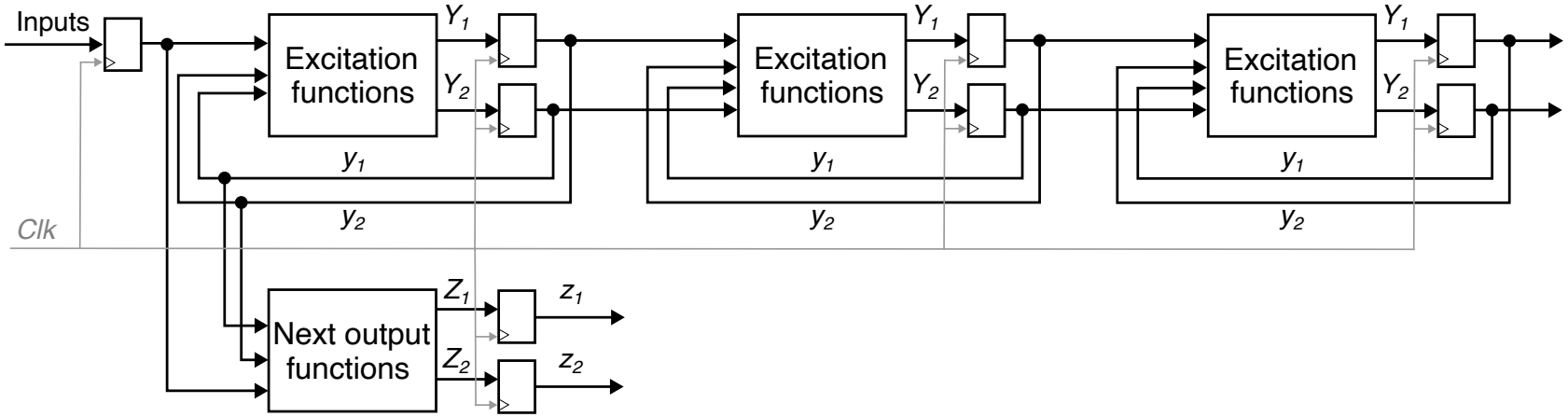
- We have mentioned that inputs must not change during transitions, and that they could possibly change simultaneously[2]
- This can be achieved with simple input registering, i.e. all inputs are stored in FFs powered with the same *Clk* signal
- We have a fully synchronous operation from inputs to outputs



[2]Within the limits of clock jitter & propagation time

Multiple logic circuits in series

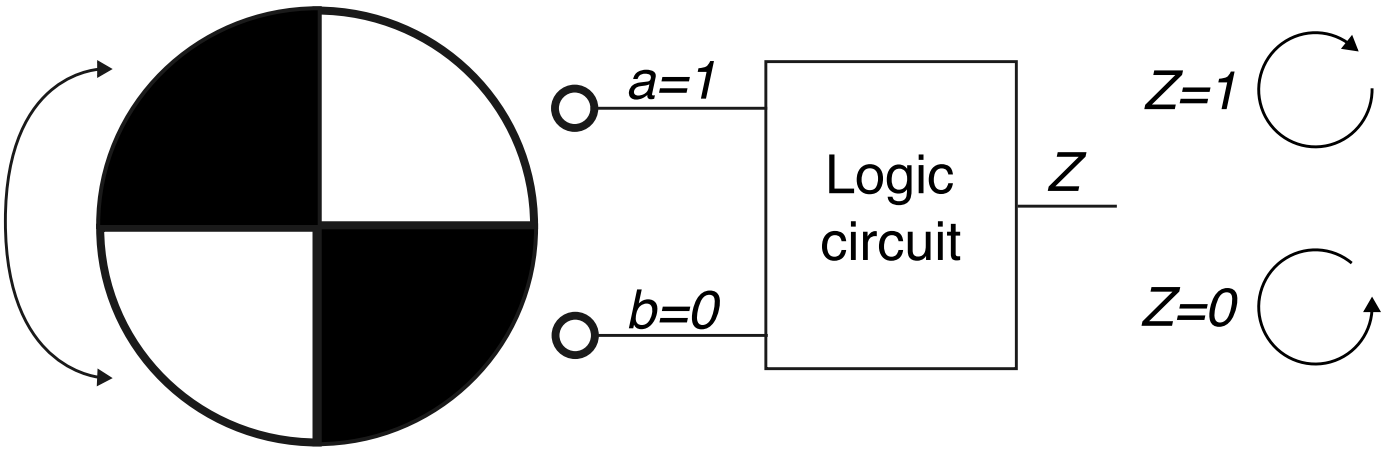
- Multiple circuits can be placed in series, outputs of ones feed the inputs of next ones – pipelined, register-to-register operation
- Note that since outputs are registered in the first sub-circuit, we can omit the FFs at the input of the next sub-circuit
- Date flows from inputs to outputs, from one circuit to another, from one set of FFs to another – Register Transfer Logic (RTL)
  - ▹ RTL paradigm is used to design logic circuits composed of tens of millions of logic gates in a single integrated circuit using CMOS process technology



5. Example of sequential system synthesis

System specification

- System is composed of a rotating axe, that can turn clock or counter clock wise. We want to design a logic system that will tell us in which direction the axe rotates. For that purpose, a disc divided into four equal segments alternatively painted in white and black is attached to the axe. In front of the disc two light sensors are placed; they can detect the color currently present on the disc: 1 for white, 0 for black. Based on color sequence that the sensors read we want to derive the rotation direction. If clock wise the output should be $Z = 1$, and $Z = 0$ otherwise.



System analysis & synthesis objectives

- The system is clearly sequential, the input alone is not sufficient to decide on the system state (rotation direction)
- We will implement sequential logic circuit system assuming:
  - Moore machine – we need to respect the output
  - Mealy machine – we don't need to respect the output
- A priori Mealy machine could be more optimal because more states could be merged, but this is not guaranteed
- Race conditions will be solved using:
  - Asynchronous logic circuit – no race conditions in the state table
  - Synchronous logic circuit – we use memories to store state values, that are updated only using a rising edge of a common control signal (Clk)
- At the end we will compare different implementations in terms of logic complexity (number of logic functions and gate count)

Primitive state table – problem analysis

- Simultaneous variation of both sensors is not possible; in another words sensor sampling is much faster then the axe rotation speed
- This means that in the primitive state table any transition from `00` to `11`, `01` to `10`, `10` to `01` and `11` to `00` will be marked with a don't care which in this particular case is a don't happen, because of the above hypothesis
- We chose for initial state `1`, input is $ab = 00$, and the axe rotates clock wise $Z = 1$
- Taking anything else as hypothesis for initial state could be correct; it is not the form of the state table that counts, but state transitions and system behaviour
- For each new state we should analyze all possible outcomes depending on the situation of the input, not that many options:
  - Either the axe changes the rotation direction
  - Or it doesn't (it keeps rotating as assumed initially)

Primitive state table construction 1/3

S^+ \ ab	00	01	11	10	Z
1	**1**	2	-	3	1
2	1	**2**		-	0
3		-		**3**	1
4					
5					
6					
7					
8					

S^-

- Initial state **1** is set for $ab = 00$
- From **1** possible evolutions:
  - ▹ Rotation direction is **opposite** to what we assumed (disc rotates counter clock wise) and the input sequence should be `00` $\rightarrow$ `01` for a newly created state **2**; output should change from '1' to '0'
  - ▹ Rotation direction is maintained, the input sequence is `00` $\rightarrow$ `10` for a newly created state **3**; output remains at '1'
- For each stable we mark don't happens: for state `1` and input $ab = 11$, for `2` and input $ab = 10$ and for `3` and input $ab = 01$

Primitive state table construction 2/3

S^+ \ ab	00	01	11	10	Z
1	**1**	2	-	3	1
2	1	**2**	4	-	0
3	5	-		**3**	1
4	-		**4**		0
5	**5**		-		0
6					
7					
8					

S^-

- From **2** rotation direction **continues** counter clock wise, the input sequence from state **1** is `00` $\rightarrow$ `01` $\rightarrow$ `11` for a newly created state **4**; output remains at '0'
- From **3** rotation direction **changes**, the input sequence is `01` $\rightarrow$ `00` we come to the initial situation but with output that should be '0' $\rightarrow$ newly created state **5**
- Don't happens are marked for state `1` and input $ab = 11$ and for state `2` and input $ab = 01$

Primitive state table construction 3/3

	ab				
S^+	00	01	11	10	Z
1	**1**	2	-	3	1
2	1	**2**	4	-	0
3	5	-	6	**3**	1
4	-	7	**4**	8	0
5	**5**	2	-	3	0
6	-	7	**6**	8	1
7	1	**7**	4	-	1
8	5	-	6	**8**	0

S^-

- Complete state table has two "parts" depending on the rotation direction
- Note state successions sequence for clock wise rotation $Z = 1$: **1**, **3**, **6**, **7**
- Note the state successions sequence for counter clock wise rotation $Z = 0$: **2**, **4**, **8**, **5**
- Once again you could pick whatever order to analyze the evolutions, this will not influence the circuit operation
- Note stable states in the same column with different outputs (important for Mealy)

A. Moore machine – state table optimization 1/3

Construction of the state equivalence table and 1st pass

S^+ \ ab	00	01	11	10	Z
1	**1**	2	-	3	1
2	1	**2**	4	-	0
3	5	-	6	**3**	1
4	-	7	**4**	8	0
5	**5**	2	-	3	0
6	-	7	**6**	8	1
7	1	**7**	4	-	1
8	5	-	6	**8**	0

S^-

2	X						
3	1-5	X					
4	X	1-7	X				
5	X	1-5	X	2-7 3-8			
6	2-7 3-8	X	3-8	X	X		
7	2-7	X	1-5	X	X	4-6	
8	X	1-5	X	4-6	3-8	X	X
	1	2	3	4	5	6	7

A. Moore machine – state table optimization 2/3

- 1-5 can't be merged, so 2-5, 2-8, 3-7 can't be merged either

2	X						
3	1-5	X					
4	X	1-7	X				
5	X	1-5	X	2-7 3-8			
6	2-7 3-8	X	3-8	X	X		
7	2-7	X	1-5	X	X	4-6	
8	X	1-5	X	4-6	3-8	X	X
	1	2	3	4	5	6	7

A. Moore machine – state table optimization 3/3

- 1-6 if 2-7 and 3-8 and 2-7 can't be merged; so 4-5 either

2	X						
3	1-5	X					
4	X	1-7	X				
5	X	1-5	X	2-7 3-8			
6	2-7 3-8	X	3-8	X	X		
7	2-7	X	1-5	X	X	4-6	
8	X	1-5	X	4-6	3-8	X	X
	1	2	3	4	5	6	7

There are no possible simplifications !!!

We will not pursue Moore implementation ...

B. Mealy machine – state table optimization

S^+ \ ab	00	01	11	10	Z
1	**1**	2	-	3	1
2	1	**2**	4	-	0
3	5	-	6	**3**	1
4	-	7	**4**	8	0
5	**5**	2	-	3	0
6	-	7	**6**	8	1
7	1	**7**	4	-	1
8	5	-	6	**8**	0

S^-

2	OK						
3	1-5	1-5					
4	3-8	2-7	4-6 3-8				
5	X	1-5	OK	2-7 3-8			
6	2-7 3-8	2-7	3-8	X	2-7		
7	2-7	X	1-5	OK	2-7	4-6	
8	1-5 3-8	1-5	X	4-6	3-8	OK	1-5
	1	2	3	4	5	6	7

B. Mealy machine – 2nd pass and fusions

2	OK						
3	1-5	1-5					
4	3-8	2-7	4-6 3-8				
5	X	1-5	OK	2-7 3-8			
6	2-7 3-8	2-7	3-8	X	2-7		
7	2-7	X	1-5	OK	2-7	4-6	
8	1-5 3-8	1-5	X	4-6	3-8	OK	1-5
	1	2	3	4	5	6	7

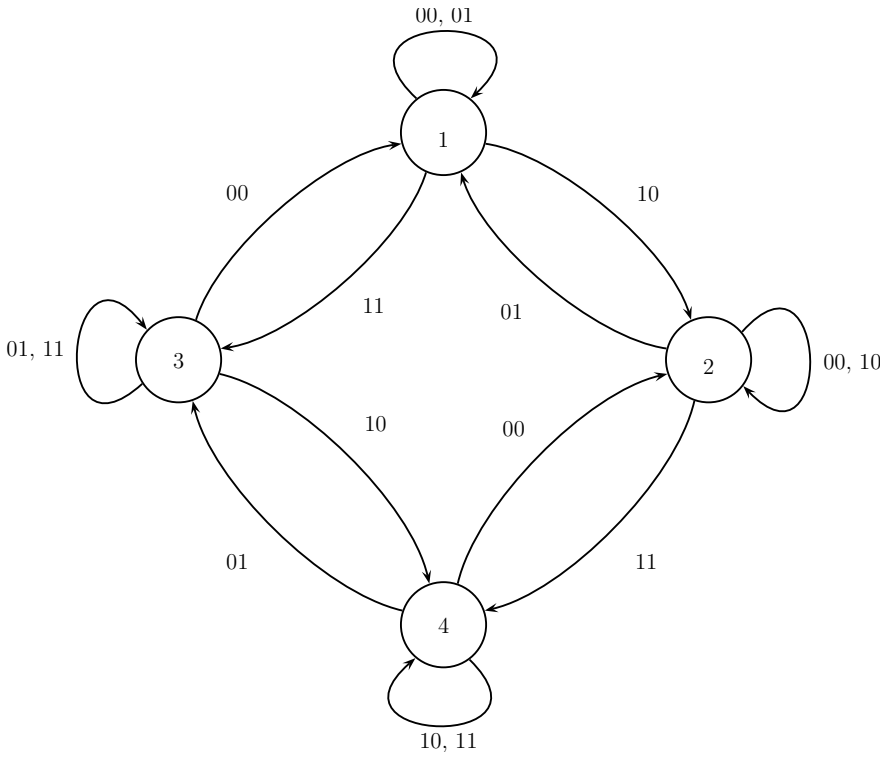
- **1** and **2** $\rightarrow$ **1**
- **3** and **5** $\rightarrow$ **2**
- **4** and **7** $\rightarrow$ **3**
- **6** and **8** $\rightarrow$ **4**

B. Mealy machine

Optimized state table and state graph

Y_2Y_1 \ ab	00	01	11	10
1	**1/1**	**1/0**	3	2
2	**2/0**	1	4	**2/1**
3	1	**3/1**	**3/0**	4
4	2	3	**4/1**	**4/0**

y_2y_1



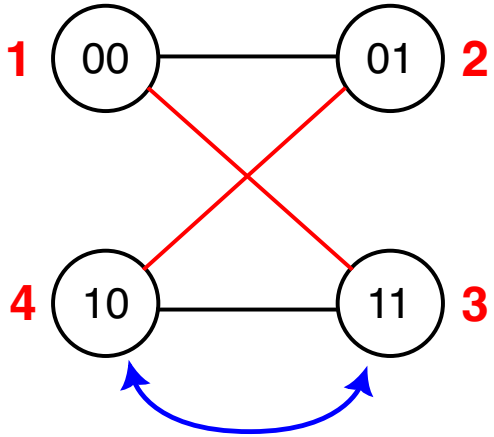
B. Mealy machine – state encoding

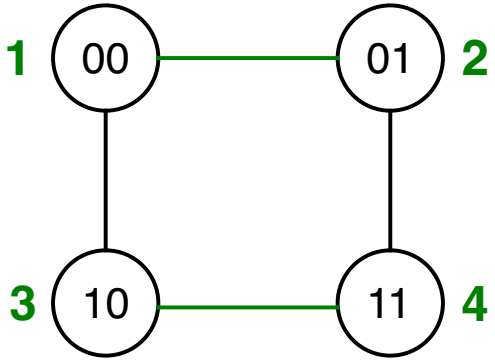
Y_2Y_1 \ ab	00	01	11	10
1	**1/1**	**1/0**	3	2
2	**2/0**	1	4	**2/1**
3	1	**3/1**	**3/0**	4
4	2	3	**4/1**	**4/0**

y_2y_1

Y_2Y_1 \ ab	00	01	11	10
00	**00**	**00**	10	01
01	**01**	00	11	**01**
11	01	10	**11**	**11**
10	00	**10**	**10**	11

y_2y_1





Attention inversion of the rows in the K-Map due to encoding

B. Mealy machine – feedback functions

Y_2Y_1 \ ab	00	01	11	10
00	**00**	**00**	10	01
01	**01**	00	11	**01**
11	01	10	**11**	11
10	00	**10**	**10**	**11**

y_2y_1 \ ab	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	1	1	1
10	0	1	1	1

y_2y_1 \ ab	00	01	11	10
00	0	0	0	1
01	1	0	1	1
11	1	0	1	1
10	0	0	0	1

$$Y_2 = y_2 b + y2a + ab$$

$$Y_1 = ab' + y_1 a + y_1 b'$$

B. Mealy machine – output synthesis

Initial table with output:

Y_2Y_1 \ ab	00	01	11	10
1	**1/1**	**1/0**	3	2
2	**2/0**	1	4	**2/1**
3	1	**3/1**	**3/0**	4
4	2	3	**4/1**	**4/0**

y_2y_1

Outputs with transitions:

Z \ ab	00	01	11	10
00	**1**	**0**	0	1
01	**0**	0	1	**1**
11	0	1	**1**	**0**
10	1	**1**	**0**	0

y_2y_1

Outputs for stable states:

Z \ ab	00	01	11	10
00	**1**	**0**		
01	**0**			**1**
11			**1**	**0**
10		**1**	**0**	

y_2y_1

y_2y_1 \ ab	00	01	11	10
00	1	0	0	1
01	0	0	1	1
11	0	1	1	0
10	1	1	0	0

States 3 & 4 are inverted!

$$Z = y_2y_1'a' + y_2y_1b + y_2'y_1a + y_2'y_1'b'$$

C. Synchronous JK – state encoding

We start with the optimized state table (Mealy) and we pick the first encoding we can think of, leaving the race conditions as they are (red colored cells in the table on the right); we can & should do this because we are implementing a synchronous circuit!!!

Initial table with output:

Y_2Y_1 \ ab	00	01	11	10
1	**1/1**	**1/0**	3	2
2	**2/0**	1	4	**2/1**
3	1	**3/1**	**3/0**	4
4	2	3	**4/1**	**4/0**

y_2y_1

Default encoding scheme:

Y_2Y_1/Z \ ab	00	01	11	10
00	**00/1**	**00/0**	11	01
01	**01/0**	00/0	10	**01/1**
11	00	**11/1**	**11/0**	10
10	01	11	**10/1**	**10/0**

y_2y_1

C. Synchronous JK – excitation table

This one is independent of the memory element choice:

Y_2Y_1/Z \ ab	00	01	11	10
00	**00/1**	**00/0**	11	01
01	**01/0**	00/0	10	**01/1**
11	00	**11/1**	**11/0**	10
10	01	11	**10/1**	**10/0**

y_2y_1

Y_2Y_1 \ ab	00	01	11	10
00	$\mu_0\mu_0$	$\mu_0\mu_0$	$\varepsilon\varepsilon$	$\mu_0\varepsilon$
01	$\mu_0\mu_1$	$\mu_0\delta$	$\varepsilon\delta$	$\mu_0\mu_1$
11	$\delta\delta$	$\mu_1\mu_1$	$\mu_1\mu_1$	$\mu_1\delta$
10	$\delta\varepsilon$	$\mu_1\varepsilon$	$\mu_1\mu_0$	$\mu_1\mu_0$

y_2y_1

C. Synchronous JK – excitation table per memory

Y_2Y_1 \ ab	00	01	11	10
00	$\mu_0\mu_0$	$\mu_0\mu_0$	$\varepsilon\varepsilon$	$\mu_0\varepsilon$
01	$\mu_0\mu_1$	$\mu_0\delta$	$\varepsilon\delta$	$\mu_0\mu_1$
11	$\delta\delta$	$\mu_1\mu_1$	$\mu_1\mu_1$	$\mu_1\delta$
10	$\delta\varepsilon$	$\mu_1\varepsilon$	$\mu_1\mu_0$	$\mu_1\mu_0$

y_2y_1

$$\Downarrow$$

M_2 \ ab	00	01	11	10
00	μ_0	μ_0	ε	μ_0
01	μ_0	μ_0	ε	μ_0
11	δ	μ_1	μ_1	μ_1
10	δ	μ_1	μ_1	μ_1

y_2y_1

M_1 \ ab	00	01	11	10
00	μ_0	μ_0	ε	ε
01	μ_1	δ	δ	μ_1
11	δ	μ_1	μ_1	δ
10	ε	ε	μ_0	μ_0

y_2y_1

C. Synchronous JK – truth tables

	J	K
μ_0	0	-
ε	1	-
δ	-	1
μ_1	-	0

ab

M_2	00	01	11	10
00	μ_0	μ_0	ε	μ_0
01	μ_0	μ_0	ε	μ_0
11	δ	μ_1	μ_1	μ_1
10	δ	μ_1	μ_1	μ_1

$y_2 y_1$

ab

M_1	00	01	11	10
00	μ_0	μ_0	ε	ε
01	μ_1	δ	δ	μ_1
11	δ	μ_1	μ_1	δ
10	ε	ε	μ_0	μ_0

$y_2 y_1$

ab

$J_2 K_2$	00	01	11	10
00	0-	0-	1-	0-
01	0-	0-	1-	0-
11	-1	-1	-0	-0
10	-1	-1	-0	-0

$y_2 y_1$

ab

$J_1 K_1$	00	01	11	10
00	0-	0-	1-	1-
01	-0	-1	-1	-0
11	-1	-0	-0	-1
10	1-	1-	0-	0-

$y_2 y_1$

C. Synchronous JK – optimized functions for M_2

J_2K_2 \ ab	00	01	11	10
00	0-	0-	1-	0-
01	0-	0-	1-	0-
11	-1	-0	-0	-0
10	-1	-0	-0	-0

y_2y_1 \ ab	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	-	-	-	-
10	-	-	-	-

$J_2 = ab$

y_2y_1 \ ab	00	01	11	10
00	-	-	-	-
01	-	-	-	-
11	1	0	0	0
10	1	0	0	0

$K_2 = a'b'$

C. Synchronous JK – optimized functions for M_1

J_1K_1 \ ab	00	01	11	10
00	0-	0-	1-	1-
01	-0	-1	-1	-0
11	-1	-0	-0	-1
10	1-	1-	0-	0-

y_2y_1 \ ab	00	01	11	10
00	0	0	1	1
01	-	-	-	-
11	-	-	-	-
10	1	1	0	0

$$J_1 = y_2a' + y_2'a$$

y_2y_1 \ ab	00	01	11	10
00	-	-	-	-
01	0	1	1	0
11	1	0	0	1
10	-	-	-	-

$$K_1 = y_2'b + y_2b'$$

C. System output

S^+/Z \ ab	00	01	11	10
00	**1/1**	**1/0**	3	2
01	**2/0**	1	4	**2/1**
11	1	**3/1**	**3/0**	4
10	2	3	**4/1**	**4/0**

S^-

Z \ ab	00	01	11	10
00	**1**	**0**	0	1
01	**0**	0	1	**1**
11	1	**1**	**0**	0
10	0	1	**1**	**0**

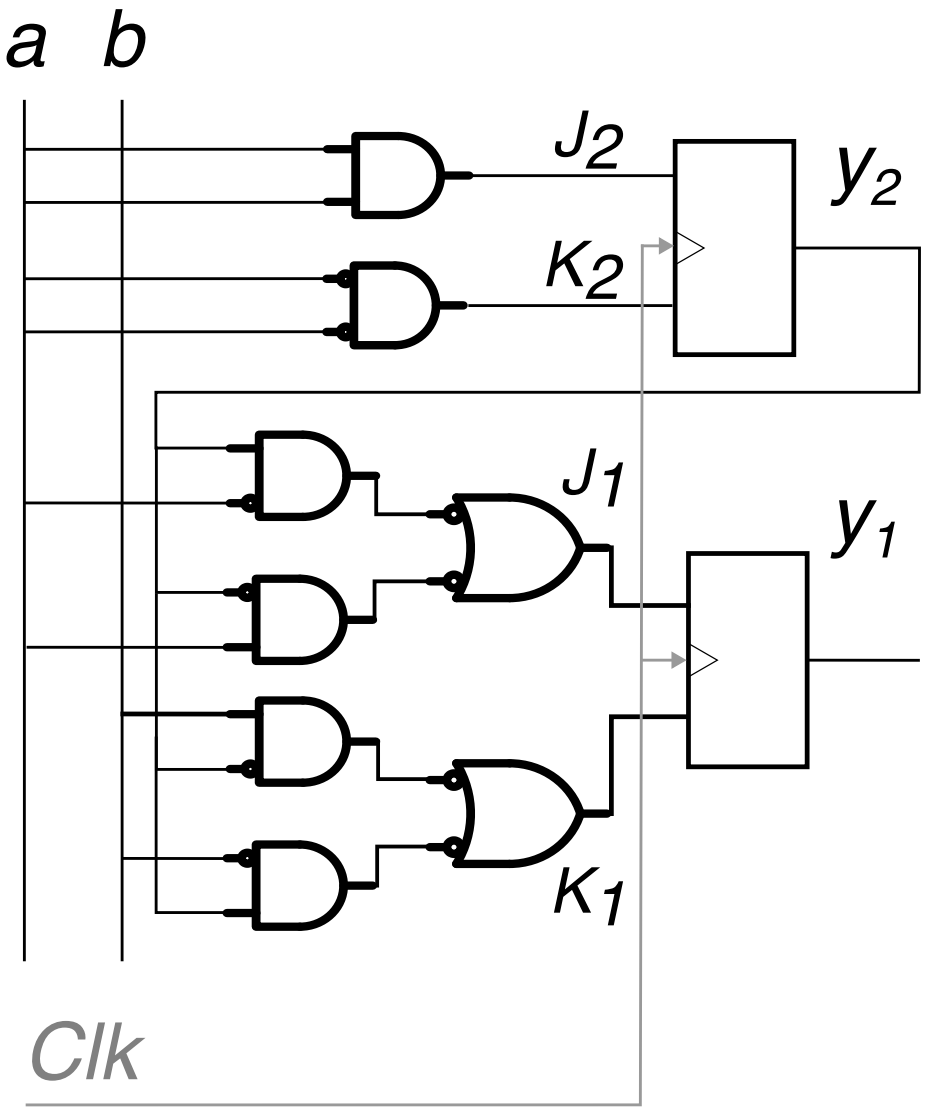
S^-

y_2y_1 \ ab	00	01	11	10
00	1	0	0	1
01	0	0	1	1
11	1	1	0	0
10	0	1	1	0

$$Z = y_2'y_1'b' + y_2'y_1a + y_2y_1a' + y_2y_1'b$$

C. Complete circuit with schematic[3]

$$
\begin{aligned}
J_2 &= ab \\
K_2 &= a'b' \\
J_1 &= y_2 a' + y_2' a \\
K_1 &= y_2' b + y_2 b' \\
Z &= y_2' y_1' b' + y_2' y_1 a \\
&+ y_2 y_1 a' + y_2 y_1' b
\end{aligned}
$$



[3] Output not shown

D. Synchronous circuit with D-FF – truth tables

	D
μ_0	0
ε	1
δ	0
μ_1	1

ab

M_2	00	01	11	10
00	μ_0	μ_0	ε	μ_0
01	μ_0	μ_0	ε	μ_0
11	δ	μ_1	μ_1	μ_1
10	δ	μ_1	μ_1	μ_1

y_2y_1

ab

M_1	00	01	11	10
00	μ_0	μ_0	ε	ε
01	μ_1	δ	δ	μ_1
11	δ	μ_1	μ_1	δ
10	ε	ε	μ_0	μ_0

y_2y_1

ab

D_2	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	1	1	1
10	0	1	1	1

y_2y_1

ab

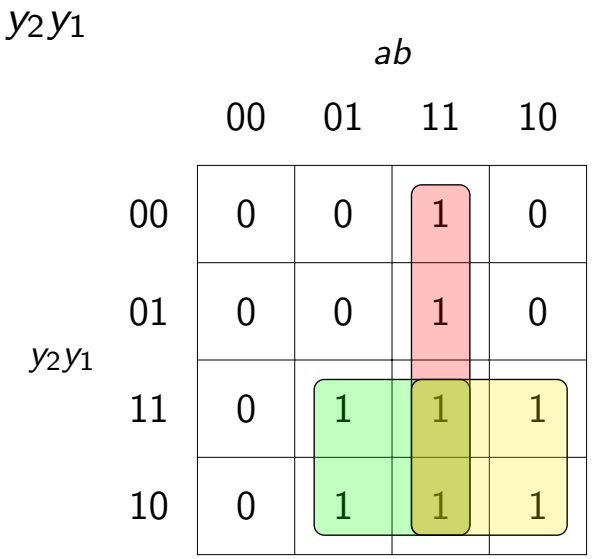
D_1	00	01	11	10
00	0	0	1	1
01	1	0	0	1
11	0	1	1	0
10	1	1	0	0

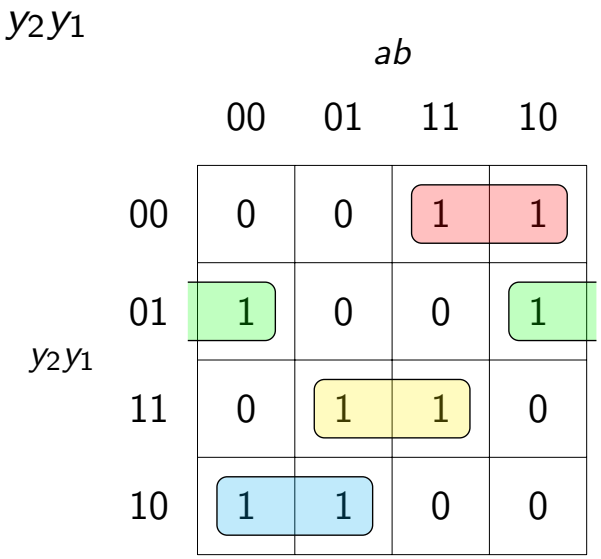
y_2y_1

D. Optimized functions for D_2 & D_1

D_2 \ ab	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	1	1	1
10	0	1	1	1

D_1 \ ab	00	01	11	10
00	0	0	1	1
01	1	0	0	1
11	0	1	1	0
10	1	1	0	0





$$D_2 = ab + y_2 b + y_2 a'$$

$$D_1 = y_2' y_1' a + y_2' y_1 b' + y_2 y_1 b + y_2 y_1' a'$$

Comparison

- Implementation A – Moore is to complex, abandoned
- Implementation B – Mealy asynchronous

$$\begin{aligned} Y_2 &= y_2 b + y2a + ab \\ Y_1 &= ab' + y_1 a + y_1 b' \\ Z &= y_2 y_1' a' + y_2 y_1 b + y_2' y_1 a + y_2' y_1' b' \end{aligned}$$

- Implementation C – Mealy synchronous with JK-FF

$$\begin{aligned} J_1 &= y_2 a' + y_2' a \qquad K_1 = y_2' b + y_2 b' \\ J_2 &= ab \qquad K_2 = a' b' \\ Z &= y_2' y_1' b' + y_2' y_1 a + y_2 y_1 a' + y_2 y_1' b \end{aligned}$$

- Implementation D – Mealy synchronous with D-FF

$$\begin{aligned} D_2 &= ab + y_2 b + y_2 a' \\ D_1 &= y_2' y_1' a + y_2' y_1 b' + y_2 y_1 b + y_2 y_1' a' \\ Z &= y_2' y_1' b' + y_2' y_1 a + y_2 y_1 a' + y_2 y_1' b \end{aligned}$$

Note on sequential system synthesis & exam

- Because of the strong dependency between different synthesis steps, we will not ask you to perform the complete sequential logic circuit synthesis in a single exercise for the exam
  - You make a slightest error in the primitive state table, everything else is wrong
- Typically we split the problem of sequential system synthesis in three independent exercises:
  - Primitive state table construction
  - State table optimization for Moore and Mealy machines
  - Synthesis of the excitation functions & output from the optimized state table
- When implementing synchronous sequential circuit don't solve race conditions before! You should leave them in the encoded state table